OBJECT CLASS

Object class is a built in class which is present inside a package called as java.lang package.

It is the supermost class of all the classes in Java

Inside Object class we have following non-static method

1) public String toString();
2) public boolean equals(Object o);
3) public int hashCode();
4) protected Object clone(); → throws ~~clone not~~ CloneNotSupportedException
5) public ~~java.lang~~ final native java.lang.Class<?> getClass();
6) public final native void notify();
7) public final native void notifyAll();
8) public final native void wait(long) throws java.lang InterruptedException;
9) public final void wait(long, int) throws InterruptedExce
10) public final void wait() throws InterruptedException;
11) protected void finalize() throws java.lang.Throwable;

1) public String toString():

toString() of Object class will return a String in the following format:

> FullyQualified Name@Hexadecimal value

The Hexa decimal value is generated with the help of integer number generated by hashCode()

NOTE: In java everytime we try to print the address of object or reference of object, the toString() of object class is implicitly called.

Eg:

```
class A
{
  int i;
  A(){}
  A(int i)
  {
  this.i=i;
  }
}
public class Driver1
    {
     public static void main(String[]args)
      {
       A obj= new A(10);
      System.out.println(obj);
      System.out.println(obj.toString());
    }
}
```

O/P:

A@15db9742

A@15db9742

o/p :

A@15db9742

A@1.5db9742.

▶ TO OVERRIDE THE toString() :

We can override toString() of Object class from subclas to print the states of an object instead of address.

E.g:

```
class Book
{
int no-pages;
String author;
double price;
Book(){}
Book(int no-pages, String author, double price)
{
  this.no-pages = no-pages;
  this.author = author;
  this.price = price;
}
```

```
public String toString()
{
System.out.println("The no. of pages:" +no-pages);
System.out.println("The author of book:" + author);
System.out.println("The price of book:" + price);
return" ";
}
```

(or)

```
public String toString()
{
return "no.of pages: " + no-pages + "; author : " + author +
        "; price of book : " + price);
}
}
class Driver2
{
public static void main (String[] args)
  {
    Book b1= new Book(100, "chetan", 35);
    System.out.println(b1);
    System.out println(b1.no-pages);
  }
}
```

Design a blue print of marker such that whenever to print a reference variable, the states of marker should print & verify it by creating min of 2 objects in driver

```
Class Marker
{
 int mid;
 double price;
 String colour;
 Marker() { }
 Marker (int mid, double price, String colour)
 {
 this.mid= mid;
 this.price = price;
 this.colour= colour;
 }
 public String toString()
 {
 return "Marker ID :"+ mid+" ; price of Marker : "+ price + ";colour
        the marker :"+ colour };
 }
 class Driver
 {
  public static void main(String[] args)
  {
   Marker m1 = new Marker (01, 10, "Blue");
   Marker m2 = new Marker (02, 10, "Black");
   System.out.println (m1);
   System.out.println (m2);
  }
 }
```

Accounts & savings account heirarchy, if we pr create account
bject, the states of a/c object should be printed whereas when
e create SA object SA states should be printed.

```
class Accounts
{
  Int AccNum;
  String AccHolder;
  String IFSC;
  Accounts() { }
  Accounts (int AccNum, String AccHolder, String IFSC)
  {
  this. AccNum = AccNum;
  this. AccHolder = AccHolder;
  this. IFSC = IFSC;
  }
  pablic String toString()
  {
  return "Marker ID:" + mid
  return "AccNum." + A
  return "Account Number :"+ AccNum + ";" Account Holder Name : " +
  AccHolder + ";IFSC code :" + IFSC ;
  }
  class SavingsAccocint extends Accounts
  {
  int balance;
double
  SavingsAccount() { }
  SavingsAccount(int AccNum, String AccHolder, String IFSC, double balance)
  {
  super(int AccNum, String AccHolder, String IFSC);
  this. balance = balance;
  }
  public String toString()
  {
    return "Account Number :"+ AccNum+ "; Account Holder Name :"-
    "AccHolder" + "; IFSC code :"+ IFSC + "; Balance : "+ balance;
```

```
}
class Driver2
{
 public static void main (String[] args)
  {
     Account a = new Account (01,"Manasa","ICICI0001");

     System.out.println(a);

     SavingsAccount sa = new SavingsAccount(02,"Rajesh","ICICI00
                                                  100000);
     System.out.println(sa);
  }
}
```

08/03/2022

▶ EQUALITY OPERATOR :

* It is a binary operator which compares LHS with RHS

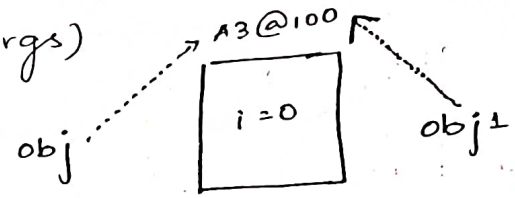
In java the equality operator compares the reference of one object with the refernce of another object.

▶ Advantage of equality operator:

1) The equality operator is used to check whether both the references are pointing to same object or different

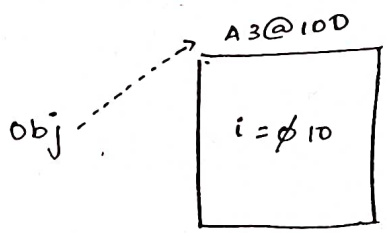
Eg:
```
class A3
{
   int i;
   A3() { }
   A3 (int i)
   {
   this.i = i;
   }
```

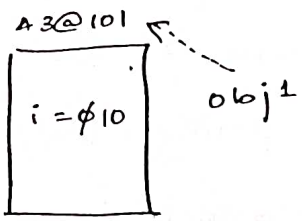
```
class Driver4
{
 public static void main (String [] args)
 {
   A3. obj = new A3 ();
   A3 obj1 = obj;
   System. out. println (obj == obj1); //true.
 }
}.
```



obj
⇓
A3@100
⇓
obj1.
(A3@100 == A3@100)
⇓
true

```
class Driver 5
{
 public static void main (String [] args)
 {
   A3 obj = new A3(10);
   A3 obj1 = new A3(10);
   System. out. println (obj == obj1);
 }
}
```





▶ public boolean equals (Object o):

The object class implementation of equal method will work just similar to equality operator. ie it will also compare the reference of two objects & returns a boole value.

CASE 1:

```
class P2
{
 int i;
 P2(){ }
 P2 (int i)
 {
 this.i=i;
 }
 public static void main (String [] args)
 {
   P2 obj = new P2(10);
  P2 obj1= new P2(10);
  System.out.println (obj);
  System.out.println (obj1);
  System.out.println (obj==obj1); //false
  System.out.println (obj.equals(obj1)); //false
 }
}
```

In the above example, even though we have created objects which are similar to one another, the equ method is returning us false, since it is comparing

where the
he address of objects has got created.

ASE2: class P3

```
{
int i;
int P3(){};
P3(int i)
{
this.i=i;
}
public static void main(String[] args)
{
P3 obj = new P3(10);
P3 obj1 = obj);
System.out.println(obj);
System.out.println(obj1);
System.out.println(obj==obj1); // true
System.out.println(obj.equals(obj1)); //true.
}
}
```

In the above example, the equals method will return as true as we have created only one object which is pointed by two reference variables. Since both the reference variables will have the same address, it will return true by comparing the references.

In both the cases, the equals method is working just similar to equality operator ie it is comparing the addresses of objects rather than comparing their

states based on which we can decide whether t
are similar objects or not. Therefore, we need to
override equals method in such a way that it
compare states of object instead of comparing the
references.

Eg:
```
class P2
{
 int i;
 P2(){}
 P2(int i)
 {
 this.i = i;
 }
 public static void main(String[] args)
 {
 public boolean equals(Object o)
 {
  P2 obj1 = (P2)o;
  return this.i = obj1.i;
 }
 public static void main(String[] args)
 {
  P2 obj = new P2(10);
  P2 obj1 = new P2(10);
  S.o.pln(obj);
  S.o.pln(obj1);
  S.o.pln(obj == obj1); //false
  S.o.pln(obj.equals(obj1)); //true.
 }
}
```

Q: class Laptop

```
{
String ram;
double price;
String brand;
Laptop() { }
Laptop (String ram, double price; String brand)
{
this. ram = ram;
this. price = price;
this. brand = brand;
}
public String toString()
{
return "Laptop RAM size : "+ ram + "; price of laptop : " +
    price + ";Brand of Laptop : "+ brand ;
}
public. boolean equals (Object O)
{
Laptop l1 = (Laptop) O;
return this.ram == l1. ram && this.price == l1. price && this. brand ==
                                                    brand;
}
}
class Driver6
{
public static void main (String [] args):
{
  Laptop l1 = new Laptop ("4 GB", 49999, "Dell");
  Laptop l2 = l1;
  System. out. println (l1 == l2);
```

```
if (l1.equals (l2))
{
  System.out.println ("Similar configuration");
 }
else
   System.out.println("Different configuration);
 }
}
```

op:

true
similar configuration.

```
class Drive17
{
 public static void main (String[] args)
 {
  Laptop l1 = new Laptop ("4 GB", 49999, "Dell");
  Laptop l2 = new Laptop ("8 GB", 49999, "Dell");
  System.out.println (l1 == l2);
  if( l1.equals(l2))
  {  System.out.println ("Similar configuration");
  else
       System.out.println ("Different configuration");
  }
 }
```

op:

false
Similar configuration.

▶ public int hashCode ():-

hashCode is an unique integer number which is created for every object created.

Object class implementation of hashcode method returns an unique integer number for each & every object created.

Eg: class P1
{
public static void main (String [] args)
{
P1 obj = new P1();
System.out.println (obj.hashCode()); // 123456
P1 obj1 = new P1();
System.out.println (obj1.hashCode()); //345678
P1 obj2 = new P1();
System.out.println (obj2.hashCode()); //569345
P1 obj3 = new P1();
System.out.println (obj3.hashCode()); //789456.
}
}

NOTE: According to Object class implementation:

1) If equals() is returning false for two object reference then their hashCode method must be returning different

2. If equals() is returning true for two object refere then their hashCode() must return same hashCode.

CASE1 : When equals() is returning false

Eg:

```
class Demo
{
int i
Demo() { }
Demo(int i)
{
this.i = i;
}
}
class Driver1
{
public static void main(String[] args)
{
Demo d1 = new Demo(10);
Demo d2 = new Demo(10);
System.out.println(d1.equals(d2));
System.out.println(d1.hashCode());
System.out.println(d2.hashCode());
}
}
```

Op:

false

123456

561234

CASE 2: When equals() is returning true:

```
class Driver2
{
 public static void main (String [] args)
 {
    Demo d1 = new Demo(10);
    Demo d2= d1;
    System.out.println (d1.equals (d2));
    System.out.println (d1.hashCode());
    System.out.println (d2.hashCode());
 }
}
```

O/P:

```
true
123456
123456
```

▶ PURPOSE OF OVERRIDING hashCode() :-

It is recommended to override hashCode() whenever equals() is overriden.

Eg:

```
class Demo1
 {
  int i;
  Demo1() { }
  Demo1(int i) {
  this.i = i;
  }
 }
  public boolean equals (Object o)
  {
  Demo1 d = (Demo1) o;
```

```
    return this.i==((Demo1)o).i;
}
public int hashcode()
{
 int hc=0;
  hc=hc+i;
  return hc;
 }
}
class Driver3
{
 public static void main(String[] args)
  {
    Demo1 d1= new Demo1(10);
    Demo1 d2= new Demo1(10);
    System.out.println(d1.equals(d2)); //true
    System.out.println(d1.hashcode()); //10
    System.out.println(d2.hashcode()); //10
  }
}
```

Eg2 :

```
class Marker
{
 int id;
 double price;
 String colour;
 Marker(){}
 Marker(int id, double price, String colour)
 {
  this.id= id;
  this.price= price;
```

```
this.colour= colour;
}

public String toString()
{
  return "Marker ID:" + id + "; price:" + price +
      "; colour of Marker:" + colour;
}
public boolean equals(Object o)
{
  Marker m = (Marker) o;
  return this.id==m.id && this.price == m.price &&
      this:colour == m.colour;
}
public int hashCode()
{
  int hc=0;
  hc= hc+id;
  hc= hc+(int)price;
  hc= hc+colour.hashCode();
  return hc;
}
}
class Drivers
{
public static void main(String[] args)
{
  Marker m1= new Marker(101, 20, "Blue");
  Marker m2= m1;
  System.out.println(m1);
  System.out.println(m2);
  System.out.println(m1.equals(m2));
  System.out.println(m2.hashCode() == m2.hashCode());
}
```

o/p:

Marker ID : 101 ; price : 20.0; colour of Marker : Blue

Marker ID : 101 ; price : 20.0; colour of Marker : Blue

true

true.

```
class Driver6
{
 public static void main (String [] args)
 {
   Marker m1 = new Marker (101, 20, "Blue");
   Marker m2 = new Marker (102, 20, "Blue");
   System.out.println (m1);
   System.out.println (m2);
   System.out.println ( m1.equals (m2));
   System.out.println ( m1.hashCode() == m2.hashCode());
   System.out.println ( m1.hashCode());
   System.out.println (m2.hashCode());
 }
}
```

o/p:

Marker ID : 101 ; price : 20.0 ; colour of Marker : Blue

Marker ID : 101 ; price : 20.0 ; colour of Marker : Blue

~~false~~ true

~~false~~ true

12345

~~67812~~ 12345

09/03/2022

STRING LITERALS :

In java, anything we pass within double quotes (" ") can be considered as string literals & strings are stored as objects in java.

To store the string literals, we can make use of following built-in classes.

1) java.lang.String.
2) java.lang.StringBuffer
3) java.lang.StringBuilder

java.lang.String:

String is a built-in-class which is present inside a package known as java.lang package. Therefore we can use string without even importing.

NOTE :

1) String class is public
2) String class is final.
3) String class extends to java.lang.Object class
4) String class implements interfaces like
   a) Comparable. interface
   b) Serializable interface
   c) CharSequence interface.

Declaration of String class:

final public class String extends Object implements Compar

Serializable, CharSequence

▶ TO CREATE OBJECTS FOR STRING:

We can create Object for strings in two ways.

1) Using String literal

2) Using new operator.

▶ USING STRING LITERAL:

Anything enclosed within double quotes is known

String literal. Everytime a String literal is passed, an

object of string is created in string Constant pool a

and its address is returned.

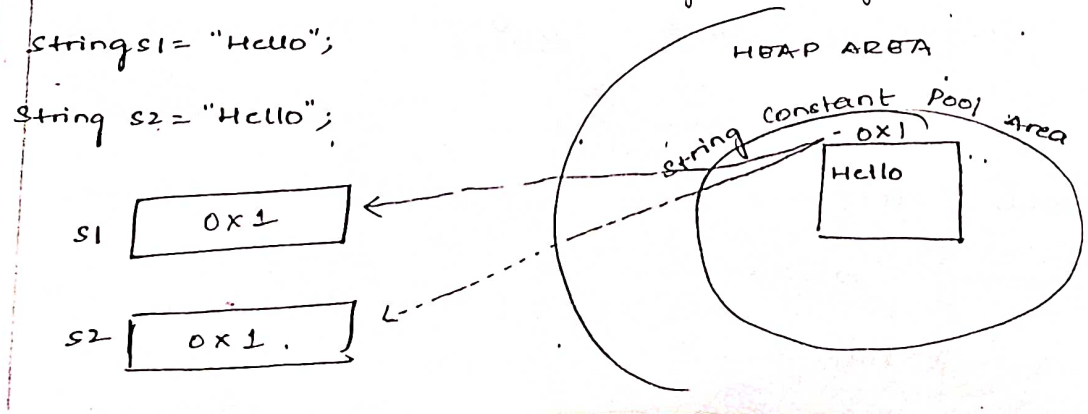
NOTE: If the string object of given literal is alread

present in the string pool area, then a new object is

not at all created, instead the address of existing

object is returned (Memory management)

```
String s1 = "Hello";
String s2 = "Hello";
```



USING NEW OPERATOR:

We can create an object for string with the help of new operator followed by constructor of the string class

CONSTRUCTORS OF STRING CLASS

1) No Argument Constructor : String ()

It is used to create an empty string object

2) String (String str) :

It is used to create a string object & initialize the object with given string.

3) String (char[] ch) :

It is used to create a string object and initialize the object with the characters present inside character array. (to convert an array of characters into a string.)

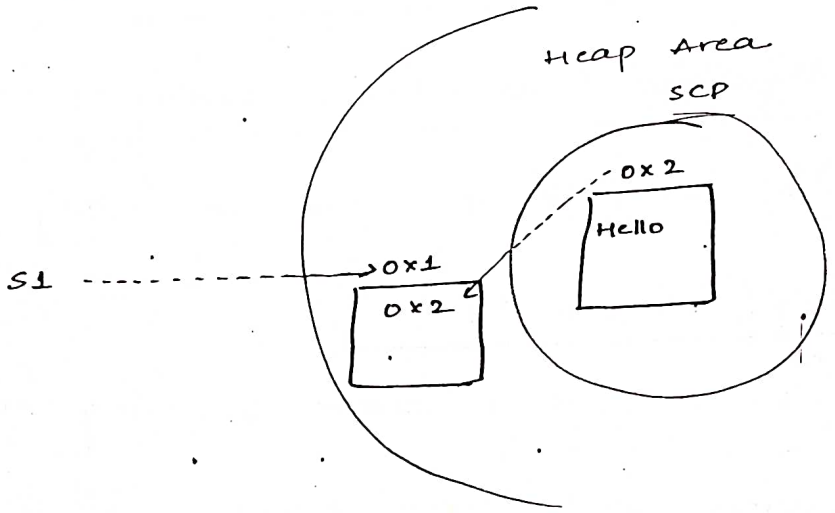
```
char[] ch = {'a', 'p', 'p'};
```

NOTE: Whenever we create an object using new operator, the object will be created inside heap area.

Eg:

```
class S1
{
public static void main (String [] args)
{
String s1 = new String("Hello");
System.out.println (s1);
}
}
```

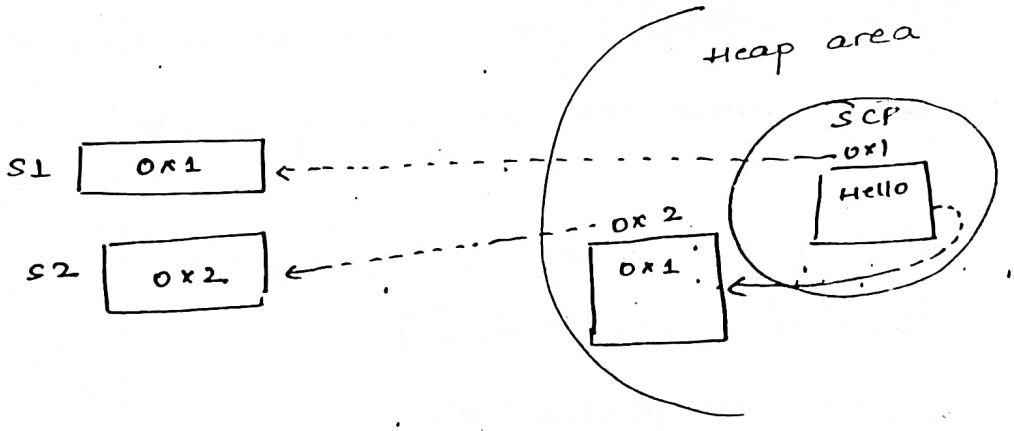
O/P:
Hello



Eg 2:

```
class S9
{
public static void main (String [] args)
{
String s1 = "Hello";
String s2 = new String ("Hello");
S.o.pln (s1);
S.o.pln (s2);
S.o.pln (s1.equals (s2));
S.o.pln (s1 == s2);
S.o.pln (s1.hashCode ());
S.o.pln (s2.hashCode ());
}
}
```

O/P: true
false
123456
123456



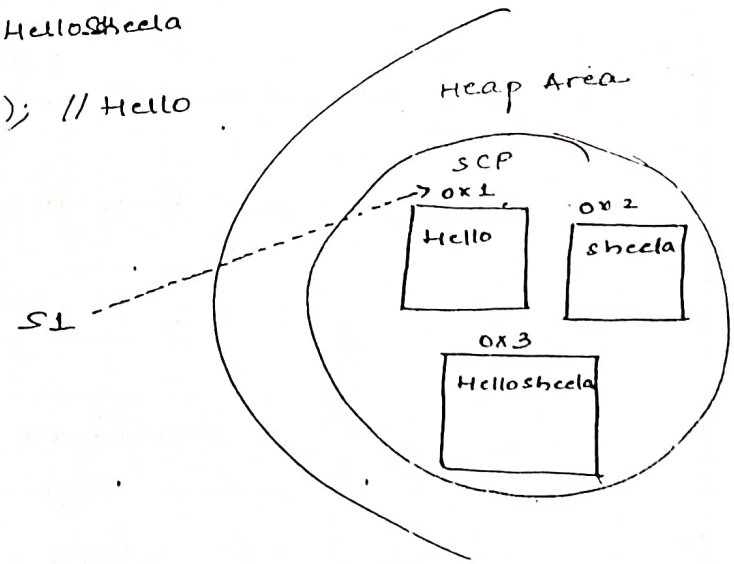
CHARACTERISTICS OF STRING:

1) String are Immutable (Not modifiable).

i.e, The states of string object once created can never be modified.

When we try to modify the string object, instead of updating the same object, it always creates a new object & its address will be returned.

Eg: class S1
{
public static void main (String [] args)
{
String s1 = "Hello";
s1.concat ("Sheela"); //HelloSheela
System.out.println (s1); // Hello
}
}

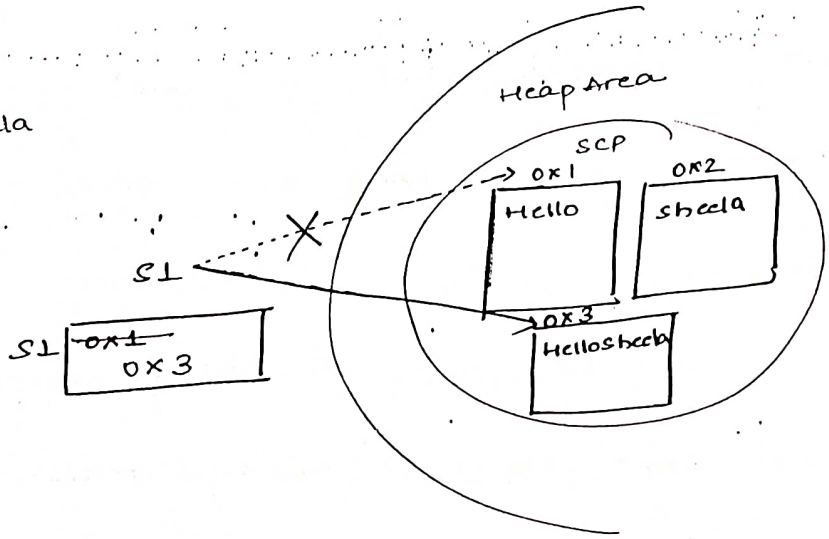


O/P:
Hello

Eg2: class S1

```
{
public static void main (String [] args)
{
  String s1 = "Hello";
  s1 = s1.concat ("sheela");
  System.out.println (s1);
}
}
```

o/p:
Hellosheela



▶ TO CONVERT CHARACTER ARRAY INTO A STRING:

We can convert a character array into a String by using following ways:

1. Using constructor of String class (String (char [] ch))
2. Using valueOf Method (array of characters is passed as input)
3. Without using built-in's.

▶ USING CONSTRUCTOR OF STRING CLASS (String(char[]ch)):

Eg:
```
class S1
{
 public static void main(String [] args)
 {
  char[] ch = {'A', 'p', 'p', 'l', 'e'};
  System.out.println(ch);
  String s=new String(ch);
  System.out.println("s : "+ s);
 }
}
```

Op:

Apple

Apple

▶ USING valueOf method(char ch):

valueOf method is a overloaded static method which is present inside String class. We can pass any of the primitive types and an array of characters etc, to the valueOf method & convert them into a String as it returns String.

SYNTAX TO CALL valueOf method:

```
String.valueOf( actual arguments );
```

Eg: class S2

```
{
public static void main (String [] args)
 {
    char[] ch = {'A', 'p', 'p', 'l', 'e'};
    System.out.println(ch);
    String s = String.valueOf(ch);
    System.out.println(s);
 }
}
```

O/P:

Apple

Apple

NOTE: We can convert any of the primitives into a String using valueOf method itself

Eg: class S3

```
{
 public static void main (String [] args)
 {
    String s = String.valueOf(10);
    System.out.println(s);
 }
}
```

O/P:

10

WITHOUT USING BUILT-IN'S :-

```
class S4
{
  public static void main (String [] args)
  {
    char [] ch = { 'b', 'a', 't' };
    String s = " ";
    for (int i = 0; i < ch.length; i++)
    {
      s += ch[i];
    }
    System.out.println (s);
  }
}
```

TO CONVERT A STRING INTO CHARACTER ARRAY :

We can convert a string into an Array of characters in two ways.

1. Using ~~to~~ toCharArray()
2. Using charAt(int index)

USING toCharArray() :

toCharArray() is a non-static method present inside String class. The return type of this method is char[].

Ex: class S5

```
{
public static void main (String [] args)
 {
   String s = "Sheela";
char[] ch = s.toCharArray()
      for (int i = 0; i < ch.length; i++)
      {
        System.out.println(ch[i]);
      }
 }
}
```

o/p:

S
h
e
e
l
a

▶ USING charAt (int index):

charAt(.) is a non-static method present inside String class. It accepts the index values & returns the character present at given Index of the string. Therefore, we can say the returntype of .charAt() is char.

NOTE: Even strings will have index values starting from zero till the length - 1, where length is not but number of characters present inside given string

length() :.

It is a non-static method present inside String class. It is used to find the number of characters present inside given String. Therefore its return type is int.

Ex:

```
class S6
{
public static void main (String [] args)
{
String s= "Apple" ;
System.out.println (s.charAt (0)); //A
System.out.println (s.charAt (1)); //p
System.out.println (s.charAt(2)); //p
System.out.println (s.charAt(4)); //e
System.out.println (s.length()); //5
}
}
```

StringIndexOutOfBoundsException:

We get StringIndexOutOfBoundsException whenever we try to access the index which is less than 0 or greater than or equal to no of characters present inside a String.

Eg: class S6

```
{
 public static void main (String [] args)
 {
  String s = "Apple";
  System.out.println (s.charAt (s.length()));//StringIndexO
                                              BoundsExcept
 }
}
```

```
class S7
{
 public static void main.(String [] args)
 {
   String s = "Manasa";
   char[] ch = new char [s.length()];
   for (int i=0 ; i< s.length() ; i++)
    {
       ch[i] = s.charAt (i);
    }
   System for (int i=0; i<ch.length ; i++)
         {
           System.out.println (ch[i]);
         }
  }
}
```

Olp:

M
a
n
a
s
a

Eg: class S6

```
{
 public static void main (String [] args)
 {
  String s = "Apple";
  System.out.println (s.charAt (s.length()));//StringIndexOutOf
                                              BoundsException
 }
}
```

```
class S7
{
 public static void main (String [] args)
 {
   String s = "Manasa";
   char[] ch = new char [s.length()];
   for (int i=0 ; i< s.length() ; i++)
    {
       ch[i] = s.charAt (i);
    }
   Syster for (int i=0; i < ch.length ; i++)
       {
         System.out.println (ch[i]);
       }
   }
 }
```

Op:

M
a
n
a
s
a

NON-STATIC METHODS OF STRING CLASS:

1) char[] toCharArray();

2) char charAt(int index);

3) length();

4) String toUpperCase():

It is used to convert a string into uppercase.

5) String toLowerCase():

It is used to convert a string into Lowercase.

6) String concat(String str):

It is used to concat a string with the given string & return the reference of concatinated string.

7) boolean contains(charSequence string):

It returns true if the given charSequence (String) is present in the String.

8) int indexOff(char ch):

It returns the position of the character, if it is present in the string. Else returns -1.

9) String substring(int start_index):

It generates a new string from the given index position upto the last character of the string.

10) int indexOf (String str):

It returns the position of given string, if it is present in the string. Else it returns -1.

11) String substring (int start-index, int end-index):

It generates a new string from the given start and end index. (end index is excluded)

12) byte[] getBytes():

It converts string into an array of bytes.

13) string[] split (string str):

It breaks a string into multiple strings and returns an array of strings.

14) boolean endsWith (String str):

It returns true if the string ends with the given string.

15) boolean startsWith (String str):

It returns true if the string starts with the given str

16) String trim():

It removes all the white spaces present in the begin and end of the string.

17) boolean equals (String str):

It is used to compare the two strings.

18) boolean equalsIgnoreCase (String str):

It returns true by comparing both the strings without considering their cases.

19) int compareTo (String str):

It is used to compare strings and return an integer.

20) int hashCode():

Assignments:

Q. WAJP to reverse the given string.

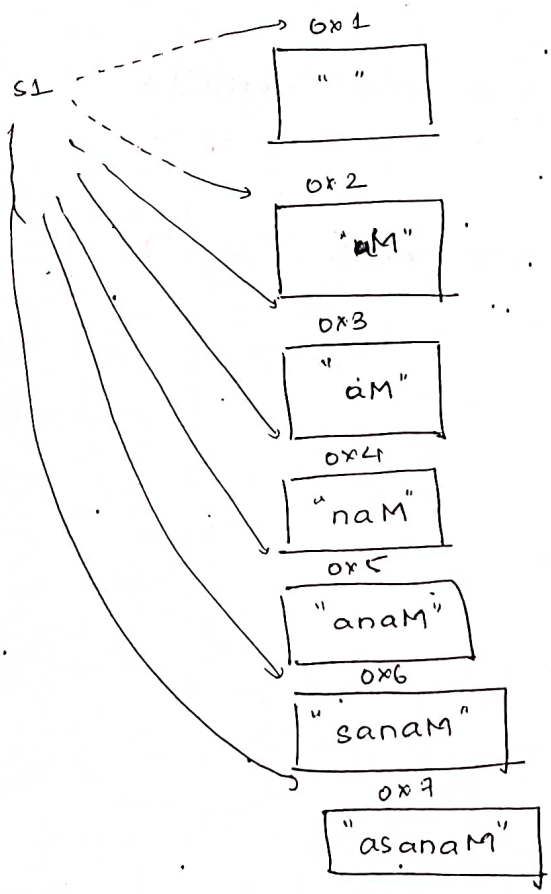
Q. WAJP to read the name of the country from the user. and also a city which belongs to the ~~red~~ country read.

Q. Print Hello if the first letter of city is present inside country name else print Bye

Q. WAJP to read the favourite colours from the user & print only the last three characters of it.

WAJP to reverse the String:

```
class RevStr
{
 public static void main (String[] args)
 {
  String s = "Manasa";
  ~~char[] ch = s.valueOf(s);~~
  char[] ch = new char[s.length()];
  String rs = "";
  for(int i=0; i<s.length(); i++)
  {
      ch[i] = s.charAt(i);
      rs = ch[i] + rs;
  }
 }
}
```



WAJP to read the name of the country from user. & also a city which belongs to read country. Print Hello if the first letter of the city is present inside country name else print Bye.

```
import java.util.Scanner;
class Name
{
  public static void main (String[] args)
  {
    Scanner sc = new Scanner (System.in);
    System.out.println ("Enter country name");
    String country = sc.nextLine();
    System.out.println ("Enter the city name");
    String city = sc.nextLine();
    char ch = s.charAt (0);
    if (country.contains (ch+" "))
    {
      System.out.println ("Hello");
    }
    else
      System.out.println ("Bye");
  }
}
```

Q. WAJP to read the favourite colours from the user & print only the last three characters of it

```
import java.util.Scanner
class Colour
{
 public static void main(String[] args)
  {
    Scanner sc= new Scanner(System.in);
    System.out.println("Enter your favourite colour");
    String colour = sc.nextLine();
    String s = colour.substring(colour.length()-3);
    System.out.println("Last three characters of colour are :" +
  }
}
```

DISADVANTAGES OF STRINGS:

Since the string objects are immutable, Everytime we try to do the modification for a string object, a new object is going to get created inside SCP. So, it will consume more amount of memory as well as time to do th modification, which reduces the performance of the application.

In order to overcome this, whenever the string involves more modifications inside an application, it is always recommended to store the string literals as StringBu -er or StringBuffer rather than a String.

STRINGBUFFER & STRINGBUILDER:-

Both the above classes can be used to store string literals. They are present inside a package known as 'java.lang' package. Therefore Fully Qualified Name of both the classes are:

→ java.lang.StringBuffer

→ java.lang.StringBuilder

▶ STRING BUILDER :

String builders cannot be used to store strings directly - that is we cannot pass string literals & store them inside String Builder container. We get compile time error.

Eg: class S1
{
public static void main (String[] args)
{
StringBuilder sb = "Hello"; //CTE
S.o.pln (sb);
}
}

How can we store String literals inside string builders?

We can store strings inside string builders with the help of constructors of stringbuilder.

▶ CONSTRUCTORS OF STRING BUILDER :-

1) StringBuilder() :

It is used to create an empty String builder object & initialize it with empty string.

The default capacity of empty string builder conta is 16.

If we try to ~~read~~ add characters more than the capacity of containers, then automatically a new container will be created implicitly with new capacity using below formula:

(previous capacity + 1) * 2

If we try to add characters more than the newly created container also, then it will not use the same formula, instead it creates a new container with the capacity same as that of number of characters added into the container.

Eg: class S3
{
public static void main (String [] args)
{
StringBuilder sb = new StringBuilder ();
System.out.println(" Before Appending :" + sb); //""
System.out println (sb.capacity ()); //16
sb.append ("abcd");
System.out.println ("After appending :"+ sb); //abcd
System.out println (sb.capacity ()); //16
sb.append ("efghijklmnopqrstuvwxyzabcdefghijklm");
System.out println ("After appending :"+ sb); //.....
System.out.println (sb.capacity ()); //39
}
}

2> StringBuilder (int capacity):

It creates a StringBuilder object with the specified capacity.

```
Ex: class S4
{
public static void main (String [] args)
{
StringBuilder sb = new StringBuilder (1000);
System.out.println (sb); //""
System.out.println (sb.capacity ()); //1000
}
}
```

3> StringBuilder (String str):

It creates a StringBuilder object & initialize it with given state. The capacity of these containers will be no of character passed during object creation + default capacity of empty string Builder object.

```
Eg: class S5
{
public static void main (String [] args)
{
StringBuilder sb= new StringBuilder ("Hello world");
System.out.println (sb); // Hello World
System.out.println (sb.capacity ()); 11+16 = 27
}
}
```

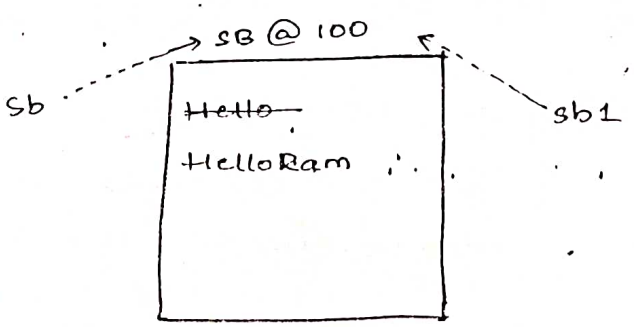
NOTE: StringBuilder and StringBuffer both are mutable whenever we try to do the modification, they will not create new object. Instead they do the modification on same object.

Eg:
```
class sb
{
public static void main (String[] args)
{
StringBuilder sb = new StringBuilder ("Hello");
StringBuilder sb1 = sb;

System.out.println ("Before Appending");
System.out.println (sb);
System.out.println (sb1);
System.out.println (sb == sb1);

sb1.append ("Ram");
System.out.println ("After Appending");
System.out.println (sb);
System.out.println (sb1);
System.out.println (sb == sb1);

}
}
```



O/P:

Before Appending
Hello
Hello
true
After Appending
Hello Ram
Hello Ram
true

2> Str...

- Inside StringBuilder only toString() is overriden.

String

-) & hashCode() are not overriden.

METHODS OF STRING BUILDER:

1. append(String str):

overloaded for all types:

append() is used to merge the given data at the last of StringBuilder object.

It is overloaded inside StringBuilder such that it can accept string, int, float, double, char[], etc., ie any data type.

2> charAt(int index)

3> length()

4> int indexOf(String):

It returns the index of String if it is present inside given String, if not it will return -1.

5> substring(int start_index):

6> substring(int start_index, int end_index)

7> reverse():

It is used to reverse the given String.

8) insert (int index, any data type):

Insert method is overloaded inside StringBuilder class, we can pass any data or we can add any type of data at a particular index by using this method.

9) setCharAt (int index, char ch) :

We can set a character at a particular index using this method.

10) trimToSize ():

It is used to reduce the capacity of StringBuilder container to the used amount of capacity

10|03|2021

WAJP to reverse a stringbuilder w/o using builtin's

```
class RevStr
{
public static void main (String[] args)
{
~~String s= "Hello";~~ StringBuilder sb = ~~"Hello";~~ new StringBuilder("Hello");
char[] ch = new char[sb.length()];
~~String rs= "";~~ StringBuilder rs = new StringBuilder("");
for(int i=0; i<sb.length(); i++);
{
  ch[i] = sb.charAt(i);
  rs = ch[i
```

```
class RevStr
{
  public static void main (String [] args)
  {
    StringBuilder sb = new StringBuilder ("Hello");
    StringBuilder rs = new StringBuilder ();
    for(int i = sb.length()-1 ; i >=0; i--)
    {
       sb1.append (sb.charAt(i));
    }
    System.out.println (sb1);
  }
}
```

1) WAJP to check whether a string is pallindrome or not.

2) WAJP to count number of upper cases, no. of lower cases, no. of spl. characters & no. of digits present inside the string which is read from the user.

3) WAP to verify the passcoord & print whether it is a valid password or not (use your own conditions for valid passcoord).

Q.1:
```
class Palindrome
{
  public static void main (String [] args)
  {
    String s = "Madam";
    String rs = "";
    char[] ch = new char [s.length()];
    ch = s.toCharArray();
    for(int i=0; i<s.length();i++)
    {
       rs = ch[i]+rs;
    }
    if(s.equals(rs))
       s.o.println("String is Palindrome");
    else
       s.o.println("String is not palindrome");
```

1) WAJP to check whether a string is pallindrome or not.

Execute & check

```
class Pallindrome
{
  public static void main (String[] args)
  {
    String s = "Madam"; String rs = "";
    char[] ch = new char[s.length()];
    ch = s.toCharArray();
    for(int i=0; i< s.length(i++)
    {
      rs = ch[i] + rs;
    }
    if (sb.equals (sb1))
    {
      System.out.println ("Given string is palindrome");
    }
    else
      System.out.println ("Given string is not a palindrome");
  }
}
```

To reverse string

2) WAJP to count number of upper cases, No. of lower cases, no. of spl characters. & not of digits present inside the string which is read from the user.

```
import java.util.Scanner;
class Count
{
  public static void main (String[] args)
  {
    Scanner sc = new Scanner (System.in);
    System.out.println ("Enter a string");
    String s = sc.nextLine ();
    int upper-case;
    int lower-case;
```

```
int spcl-char;
int digit;
for (int i=0; i< sb.length() ;i++)
 {
    char ch = sb.charAt(i);
    if ( ch >= 'A' && ch<= 'Z')
        upper-case++;
    else if ( ch >='a' && ch<='z')
        lower_case++;
    else if (ch >='0' && ch<='9')
        digit++;
    else
        spcl-char++;
 }
System.out.println ("Number of uppercases in string :"+ upper-case
System.out.println ("Number of lower cases in string :"+ lower-case
System.out.println ("Number of special characters in string :" +
                                        spcl-char);
System.out.println ("Number of digits in string :" + digits);
}
}
```

3) WAJP to verify the password & print whether it is a valid password or not

```
import java.util.Scanner;
Class Password
{
    public static void main (String[] args)
    {
     Scanner sc= new Scanner(System.in);
     System.out.println ("Enter password");
     String pwd= sc.nextLine();

     int upper-case;
     int lower-case;
     int digit;
     int spcl-char;
     for(int i=0; i< pwd.length(); i++)
      {
         char ch = sb.charAt(i);
         if (ch>='A' && ch<='Z');
             upper-case++;
       elseif (ch>='a' && ch<='z');
             lower-case++;
       else if (ch>='0' && ch<='9');
             digit++;
         if else
             spcl-char++;
      }
    if (pwd.length()>=8 && upper-case>0 && lower-case>0 && digit>0 &&
                  spcl-char>0)
      {
       System.out.println("password entered is valid");
      }
  else
     System.out.println("password entered is invalid");
}
}
```